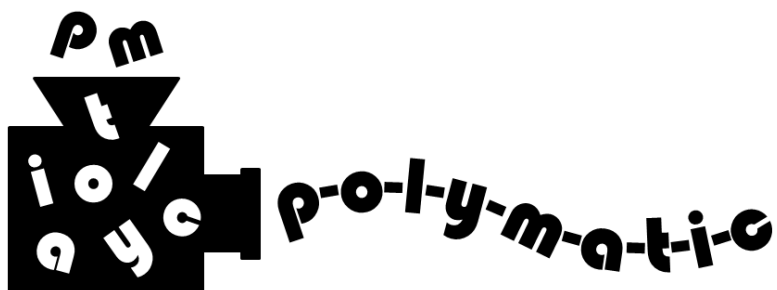


POLYMATIC USER MANUAL

Version 1.1

August 16, 2015



The Colina Group
Department of Materials Science and Engineering
The Pennsylvania State University

<http://www.matse.psu.edu/colinagroup/polymatic>
<https://nanohub.org/resources/17278>

Copyright © 2013, 2015 Lauren J. Abbott
Distributed under the terms of the GNU General Public License

Contents

1	Introduction	3
1.1	About <i>Polymatic</i>	3
1.2	Included Files	3
1.3	Version History	4
1.4	GNU Distribution Notice	5
1.5	Citation	5
1.6	Publications Using Polymatic	5
1.7	Acknowledgments	7
2	Random Packing	9
2.1	About	9
2.2	Syntax	10
2.3	Output	10
2.4	Usage	11
2.5	Examples	13
3	Simulated Polymerization	15
3.1	About	15
3.2	Polymerization Step	17
3.2.1	About	17
3.2.2	Syntax	17
3.2.3	Types File	18
3.2.4	Input Script	19
3.2.5	Output	25
3.3	Initialization and Finalization	26
3.3.1	About	26
3.3.2	Syntax	26
3.3.3	Output	27
3.4	Polymerization Loop	27
3.4.1	About	27
3.4.2	Syntax	28
3.4.3	Parameters and Input Scripts	28
3.4.4	Setup and Output	30
3.5	Usage	32
3.6	Examples	34

A	Data Structures	39
A.1	About	39
A.2	Molecular System	39
A.2.1	Flags	40
A.2.2	Periodic Box	40
A.2.3	Neighbor List	41
A.2.4	Molecules	41
A.2.5	Atoms	42
A.2.6	Bonded Interactions	43
A.2.7	Data Types	44
A.3	Input Script Parameters	47
A.3.1	Link Command	47
A.3.2	Charge Command	48
A.3.3	Connect Command	48
A.3.4	Types Command	49
A.3.5	Bond Command	49
A.3.6	Align Command	50
B	File Conversion	51
B.1	About	51
B.2	Reading and Writing Files	51
B.3	Connectivity Definitions	52
B.4	Usage	53

1 Introduction

1.1 About *Polymatic*

Polymatic is a code developed in the [Colina Group](#) for structure generation of amorphous polymers by a simulated polymerization algorithm. *Polymatic* can be thought of as a wrap-around code that calls a simulation package to perform energy minimization and molecular dynamics simulations. In between these simulations, the code performs polymerization steps by checking a set of defined bonding criteria and updating the connectivity information of the system (i.e., bonds, angles, dihedrals, impropers) appropriately to reflect the new bonds being made. Codes are included to perform an initial random packing (Chapter [2](#)) and polymerization steps (Chapter [3](#)).

The current *Polymatic* distribution is set up to work with the Large-scale Atomic/Molecular Massively Parallel Simulator ([LAMMPS](#)). Specifically, it is set up to read and write LAMMPS [data files](#) with a class I or II [force field](#), including pair, bond, angle, dihedral, and improper terms. However, the majority of the tasks within the codes do not rely on these LAMMPS definitions, only on proper definition of the molecular systems (Appendix [A](#)). With that in mind, some additional codes have been included in this distribution to read and write other popular file types to aid in the extension of *Polymatic* to other file types and software packages (Appendix [B](#)).

1.2 Included Files

The distribution of *Polymatic* includes the following files:

1. `polymatic_manual.pdf` (this file), a user manual for *Polymatic*
2. `Polymatic.pm`, a perl module with subroutines to read and write different file types and perform common tasks on molecular systems
3. `pack.pl`, a perl script that performs a random packing of molecules in a periodic cubic box
4. `polym.pl`, a perl script that performs a single polymerization step

5. `polym_init.pl`, a perl script that performs an initialization of the system before polymerization
6. `polym_final.pl`, a perl script that performs a finalization of the system after polymerization
7. `polym_loop.py`, a python script to control the workflow of the simulated polymerization loop
8. `file_convert.pl`, a perl script to convert file types
9. `Example*/`, directories containing all files for the examples included in this manual (input and output)
10. `COPYING`, a text file of the GNU General Public License

1.3 Version History

v1.1: Released August 16, 2015.

- Changed how molecular systems are stored to improve usability. Now using a hash table to represent a single molecular system.
- Changed how the polymerization workflow is handled. Now using a python script to replace the bash script from the previous version.
- Changed how some of the subroutines are defined. Now using a perl module to prevent repetitive definitions, which handles file input and output and basic tasks on molecular systems.
- Added the ability to handle class I force fields, in addition to class II force fields. Primarily affects file input and output.
- Added the ability to perform the random packing code starting with an existing structure.
- Added new functionality for intramolecular bonding for networks.
- Added new codes to help with file conversion between several popular molecular file types.
- Other minor bug fixes and changes to the code structure.

v1.0: Released March 26, 2013.

1.4 GNU Distribution Notice

Polymatic is free software: you can redistribute it and/or modify it under the terms of the GNU [General Public License](#) as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Polymatic is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with *Polymatic*. If not, see <http://www.gnu.org/licenses/>.

1.5 Citation

The following paper describes the basic *Polymatic* algorithm and its implementation for a variety of linear polymers. If you publish work using *Polymatic* or variations of it, please cite this paper and the source code:

Abbott, L. J.; Hart, K. E.; Colina, C. M. “Polymatic: A Generalized Simulated Polymerization Algorithm for Amorphous Polymers.” *Theor. Chem. Acc.*, **2013**, *132*, 1334. DOI: [10.1007/s00214-013-1334-z](https://doi.org/10.1007/s00214-013-1334-z)

Abbott, L. J.; Colina, C. M. *Polymatic: A Simulated Polymerization Algorithm*, Version 1.1, **2015**, <https://nanohub.org/resources/17278>.

1.6 Publications Using Polymatic

Below is a growing list of published work that uses *Polymatic* or variations of it (let us know if you have a paper to add!):

1. Abbott, L. J.; Colina, C. M. Atomistic Structure Generation and Gas Adsorption Simulations of Microporous Polymer Networks. *Macromolecules*, **2011**, *44*, 4511–4519. [\[DOI\]](#)

2. Abbott, L. J.; Hart, K. E.; Colina, C. M. Polymatic: A Generalized Simulated Polymerization Algorithm for Amorphous Polymers. *Theor. Chem. Acc.*, **2013**, *132*, 1334. [\[DOI\]](#)
3. Hart, K. E.; Abbott, L. J.; Colina, C. M. Analysis of Force Fields and BET Theory for Polymers of Intrinsic Microporosity. *Mol. Simul.*, **2013**, *39*, 397–404. [\[DOI\]](#)
4. Hart, K. E.; Abbott, L. J.; Colina, C. M. Toward Effective CO₂/CH₄ Separations by Sulfur-Containing PIMs via Predictive Molecular Simulations. *Macromolecules*, **2013**, *46*, 5371–5380. [\[DOI\]](#)
5. Zhou, X.; Li, Y.; Hart, K. E.; Abbott, L. J.; Lin, Z.; Svec, F.; Colina, C. M.; Turner, S. R. Nanoporous Structure of Semirigid Alternating Copolymers via Nitrogen Sorption and Molecular Simulation. *Macromolecules*, **2013**, *46*, 5968–5973. [\[DOI\]](#)
6. Perkins, S. L.; Painter, P.; Colina, C. M. Molecular Dynamic Simulations and Vibrational Analysis of an Ionic Liquid Analogue. *J. Phys. Chem. B*, **2013**, *117*, 10250–10260. [\[DOI\]](#)
7. Abbott, L. J.; McKeown, N. B.; Colina, C. M. Design Principles for Microporous Organic Solids From Predictive Computational Screening. *J. Mater. Chem. A*, **2013**, *1*, 11950–11960. [\[DOI\]](#)
8. Hart, K. E.; Springmeier, J. M.; McKeown, N. B.; Colina, C. M. Simulated Swelling During Low-Temperature N₂ Adsorption in Polymers of Intrinsic Microporosity. *Phys. Chem. Chem. Phys.*, **2013**, *15*, 20161–20169. [\[DOI\]](#)
9. Abbott, L. J.; Hughes, J. E.; Colina, C. M. Virtual Synthesis of Thermally Cross-Linked Copolymers from a Novel Implementation of Polymatic. *J. Phys. Chem. B*, **2014**, *118*, 1916–1924. [\[DOI\]](#)
10. Abbott, L. J.; Colina, C. M. Porosity and Ring Formation in Conjugated Microporous Polymers. *J. Chem. Eng. Data*, **2014**, *59*, 3177–3182. [\[DOI\]](#)
11. Abbott, L. J.; Colina, C. M. Formation of Microporosity in Hyper-Cross-Linked Polymers. *Macromolecules*, **2014**, *47*, 5409–5415. [\[DOI\]](#)

12. Hart, K. E.; Colina, C. M. Estimating Gas Permeability and Permselectivity of Microporous Polymers. *J. Membr. Sci.*, **2014**, *468*, 259–268. [\[DOI\]](#)
13. Hart, K. E.; Colina, C. M. Ionomers of Intrinsic Microporosity: In Silico Development of Ionic-Functionalized Gas-Separation Membranes. *Langmuir*, **2014**, *30*, 12039–12048. [\[DOI\]](#)
14. Hart, K. E.; Abbott, L. J.; Lísal, M.; Colina, C. M. Morphology and Molecular Bridging in Comb- and Star-Shaped Diblock Copolymers. *J. Chem. Phys.*, **2014**, *141*, 204902. [\[DOI\]](#)
15. Jin, K.; Qin, Z.; Buehler, M. J. Molecular Deformation Mechanisms of the Wood Cell Wall Material. *J. Mech. Behav. Biomed. Mater.*, **2015**, *42*, 198–206. [\[DOI\]](#)
16. Zhou, X.; Huang, J.; Barr, K. W.; Lin, Z.; Maya, F.; Abbott, L. J.; Colina, C. M.; Svec, F.; Turner, S. R. Nanoporous Hypercrosslinked Polymers Containing T_g Enhancing Comonomers. *Polymer*, **2015**, *59*, 42–48. [\[DOI\]](#)
17. Frentrop, H.; Hart, K. E.; Colina, C. M.; Müller, E. A. In Silico Determination of Gas Permeabilities by Non-Equilibrium Molecular Dynamics: CO₂ and He through PIM-1. *Membranes*, **2015**, *5*, 99–119. [\[DOI\]](#)

1.7 Acknowledgments

Below is a list of funding sources, computational support, and people that have contributed to the development of *Polymatic* in some way:

- Funding was provided by the [National Science Foundation](#) (NSF) with grant DMR-0908781.
- Computational resources and support were provided in part by Penn State Information Technology Services.
- Members of the Colina group provided valuable comments and suggestions during the development of the algorithm and code.

- Kyle E. Hart was heavily involved in the testing and development of the code since its early stages and has provided many helpful comments and suggestions for improving its functionality and usability.
- Michael E. Fortunato suggested a python workflow script, which was added in version 1.1. He has also developed a web-based GUI tool [nuSIMM](#) that implements *Polymatic*.
- Grant Gonzalez and Justin E. Hughes were undergraduate researchers in the Colina group who worked with the code and helped with testing at different stages of the development.
- Jeff Thompson provided useful feedback and suggestions for improving the usability and functionality of the code.

2 Random Packing

2.1 About

The random packing code included with this distribution (`pack.pl`) packs any number of reference molecules into a periodic cubic simulation box. A specified number of each reference molecule is packed into the periodic box one at a time, where insertions are made avoiding only hard-core overlaps of the atoms. For ease of use with the *Polymatic* simulated polymerization code, the packing code reads and writes LAMMPS data files.

Users should note that the intention of the packing code is to provide initial systems of molecules and/or polymeric repeat units at low densities, such as for use with the *Polymatic* simulated polymerization code. It was not designed to handle packing of large and complex molecules at high densities, to introduce flexibility into the original reference molecule structure (e.g., using Monte Carlo moves), nor to place any restrictions on regions in which to pack the molecules.

Note: The packing code is not required to generate initial structures for the *Polymatic* simulated polymerization. Some other examples of software that can perform packing and building tasks include:

1. [Packmol](#)
2. [Enhanced Monte Carlo](#)
3. [MCCCS Towhee](#)
4. [GROMACS](#)
5. [Polymer Modeler](#)
6. [BIOVIA Materials Studio](#)
7. [Scienomics MAPS](#)

2.2 Syntax

The syntax of the random packing code is:

```
./pack.pl -i num F1.lmps N1 F2.lmps N2 (...)  
          -l boxL  
          -o pack.lmps
```

Below are descriptions of the command-line flags:

1. The `-i` flag indicates the reference molecules (e.g., `F1.lmps`) and the number of each to be packed in the periodic cubic box (e.g., `N1`). The number of reference molecules to be read in is indicated by `num`. To pack into an existing structure keeping its atom coordinates, it should be given as the first reference file (`F1.lmps`) with `N1` set to 0.
2. The `-l` flag defines the length of the sides of the cubic simulation box. If an existing initial structure is being used, its box dimensions can be maintained by setting `boxL` to 0, otherwise the box is resized with the given `boxL` value. In this case, only the box dimensions are changed, while the atom coordinates are kept unchanged.
3. The `-o` flag gives the name of the output file to be created containing the packed molecules.
4. The `-h` flag prints the syntax, then quits the program.

Note: The order in which the flags are specified does not matter, but the order of the arguments for each individual flag does.

2.3 Output

After packing is complete, a LAMMPS data file is created containing the new system (`pack.lmps`). Additionally, while the code is running, the progress of the packing is printed to standard output. A header line is printed for each new reference molecule with the file name and number of molecules to be packed. After each successful insertion, the molecule number is printed along with the number of attempts required for that molecule.

For example, a packing of 3 molecules of one reference type and 2 molecules of a second reference type would generate output similar to the following:

```
Packing 3 molecules of reference 'example1.lmps':  
  1, 1 attempts  
  2, 1 attempts  
  3, 4 attempts  
Packing 2 molecules of reference 'example2.lmps':  
  1, 2 attempts  
  2, 5 attempts
```

Note: If an existing initial structure is used as the first reference file, it will be printed as packing 0 molecules.

2.4 Usage

Below is a list of notes about the packing code that may be useful for using and/or modifying it:

1. Some subroutines in the `Polymatic.pm` module are used by the packing code. It therefore needs to be included in the same directory or a path to its directory should be defined (e.g., with `use lib`).
2. If more than one reference molecule is provided, all molecules of the first reference type are packed first, followed by the next, and so on in the order listed. For most efficient performance, it is advised that the larger molecule types are packed first.
3. If more than one reference file is being used, numeric data types are assumed to be the same for all files. These types are also carried over to the newly generated system.
4. To use an existing initial structure during the random packing, it should be listed as the first reference molecule with the number of molecules to pack set to 0. In this case, if new box dimensions are specified, only the box size is changed, while the atom positions are kept the same. Currently, only cubic boxes are supported.

5. A maximum number of insertion attempts for each molecule is defined to prevent the code from running in an infinite loop. This parameter is defined as `$maxAttempts` at the top of the file.
6. Test insertions are rejected if two atoms are closer than $s * (r_i + r_j)$, where r is the radius of the atom and s is a scaling parameter. The amount of overlap allowed can be adjusted by the scaling parameter, which is defined as `$scale` at the top of the file.
7. If Lennard–Jones parameters are not defined in the reference files, a default radius is used for all atoms. This parameter is defined as `$radius` at the top of the file.
8. The `defineSystems` subroutine reads in the reference molecules and stores each as an individual molecular system hash in the `@refs` array. It also initializes the new packing system, which is stored as a new molecular system in the `%sys` hash.
9. The `packMols` subroutine is the main loop controlling the packing. It calls the `centerMol` subroutine to move the reference molecule to the origin. In iterations, a random molecule is then generated in the `generateRandomMol` subroutine. If no overlap with other atoms in the system is found in the `checkOverlap` subroutine, then it is added to the packing system in the `addMol` subroutine.
10. Files are read and written in the LAMMPS data format with calls to the `readLammps` and `writeLammps` subroutines in the `Polymatic.pm` module. This would be the place to start in order to adapt the code to work with other file types (such as those included in the `Polymatic.pm` module). It may also be necessary to update what information is stored in the new system as it is being created, as different file types include different information.

Note: The *Polymatic* distribution uses hashes to store information defining a molecular system, which is discussed in Appendix A. In addition, the `Polymatic.pm` module can read and write several popular file types to and from molecular system hashes, as described in Appendix B. This information may be useful if modifying the codes.

2.5 Examples

The input and output files for each example are provided with the *Polymatic* distribution in the corresponding **Example*** directory. Illustrations for each are shown in Figure 1.

Example 1: This example packs 45 repeat units of polystyrene (reference file `ps_monomer.lmps`) into a periodic simulation box with length 30 Å. The packed system created is written to `pack.lmps`.

```
./pack.pl -i 1 ps_monomer.lmps 45 -l 30 -o pack.lmps
```

Example 2: This example packs 6 molecules of C₆₀ fullerene and 40 repeat units of polycarbonate (reference files `c60.lmps` and `pc_monomer.lmps`) into a periodic simulation box of length 40 Å. The packed system created is written to `pack.lmps`.

```
./pack.pl -i 2 c60.lmps 6 pc_monomer.lmps 40 -l 40 \  
-o pack.lmps
```

When using more than one reference molecule type, as in this example, it is important to ensure that the data types are consistent for all files, even if not all data types are used in a file.

Example 3: This example packs 150 molecules of water (reference file `water.lmps`) into an existing periodic simulation box with 45 repeat units of polystyrene (`ps_pack.lmps`, from Example 1), maintaining its box dimensions. The packed system created is written to `pack.lmps`.

```
./pack.pl -i 2 ps_pack.lmps 0 water.lmps 150 -l 0 \  
-o pack.lmps
```

For this example, the data types required for the water molecules were added to the polystyrene packed box from Example 1.

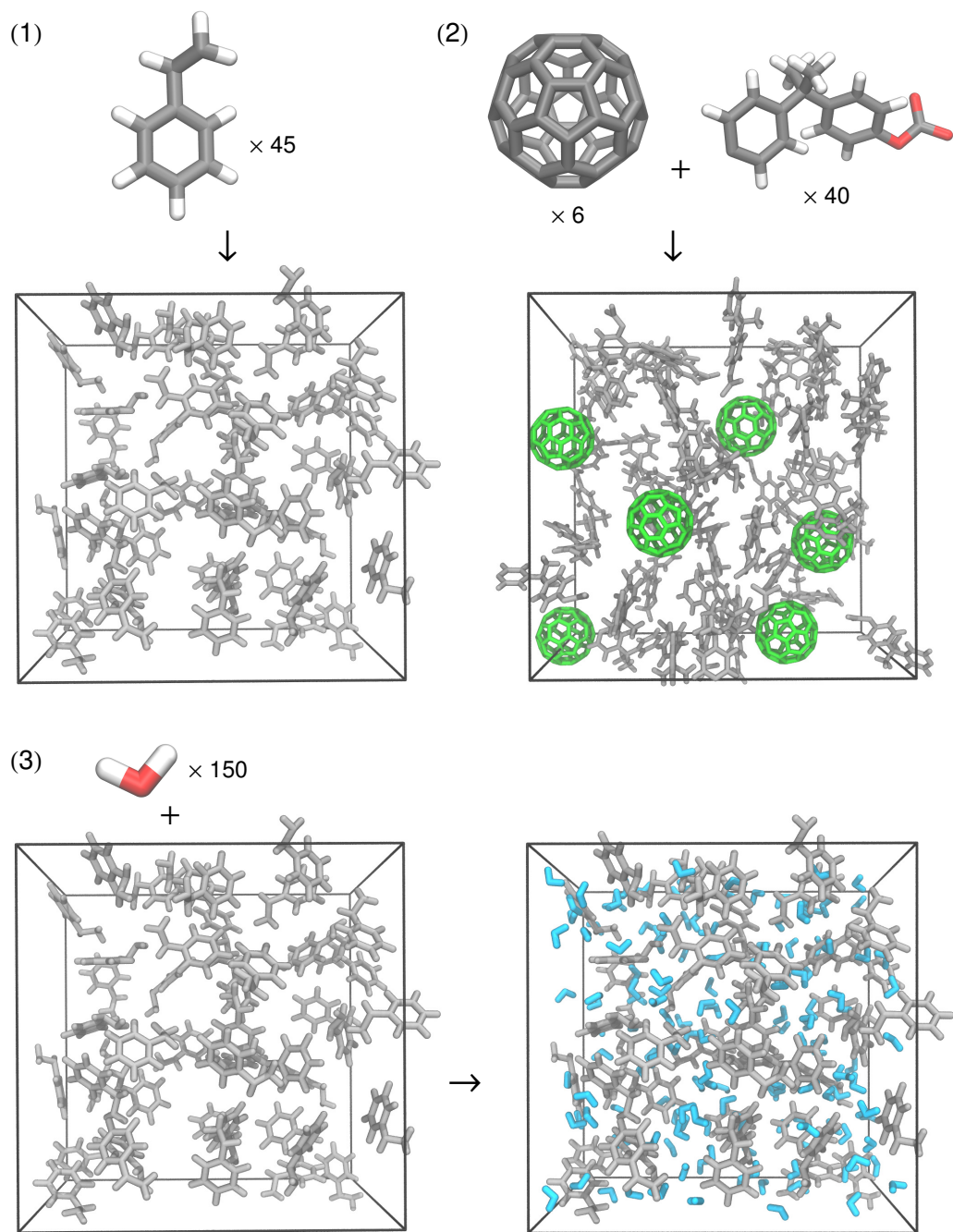


Figure 1. Illustrations of Examples 1–3.

3 Simulated Polymerization

3.1 About

The *Polymatic* distribution includes codes to perform a simulated polymerization of a simulation box of monomers or repeat units. Using this approach, repeat units in close proximity are bonded according to a set of criteria, which take place along with energy minimization and molecular dynamics simulations in cycles to form a final polymeric system. A flowchart of the algorithm is given in Figure 2. It can be grouped into five main components:

1. *Polymerization initialization*: An initial system to be polymerized is provided, with the linking atoms identified by unique atom types. Optionally, artificial charges can be added to linking atoms to encourage more efficient bonding in the polymerization process. (Section 3.3)
2. *Polymerization step*: The closest pair of linking atoms that satisfies all bonding criteria is selected, bond(s) are formed to obtain the proper polymeric structure, and an energy minimization is performed. If artificial charges were added, they are removed from the pair of bonded linking atoms. If no pair meeting all bonding criteria is found, a molecular dynamics (MD) simulation is performed and a polymerization step attempted again. Up to $M_{\max}$ polymerization steps are attempted, after which the polymerization loop (see below) is quit. (Section 3.2)
3. *Polymerization cycle*: N_{cyc} polymerization steps are performed. Then, an MD step is carried out to allow for relaxation and rearrangement of the structure. This composes a polymerization cycle. Multiple MD types can be defined and alternated throughout; in this algorithm, a second MD type is implemented every N_{md} cycles. (Section 3.4)
4. *Polymerization loop*: Polymerization cycles are repeated until B_{tot} bonds are formed or until no pair meeting the bonding criteria is identified in $M_{\max}$ attempts. (Section 3.4)
5. *Polymerization finalization*: The system is finalized to complete the polymerization. If artificial charges were added, they are removed from any remaining linking atoms. (Section 3.3)

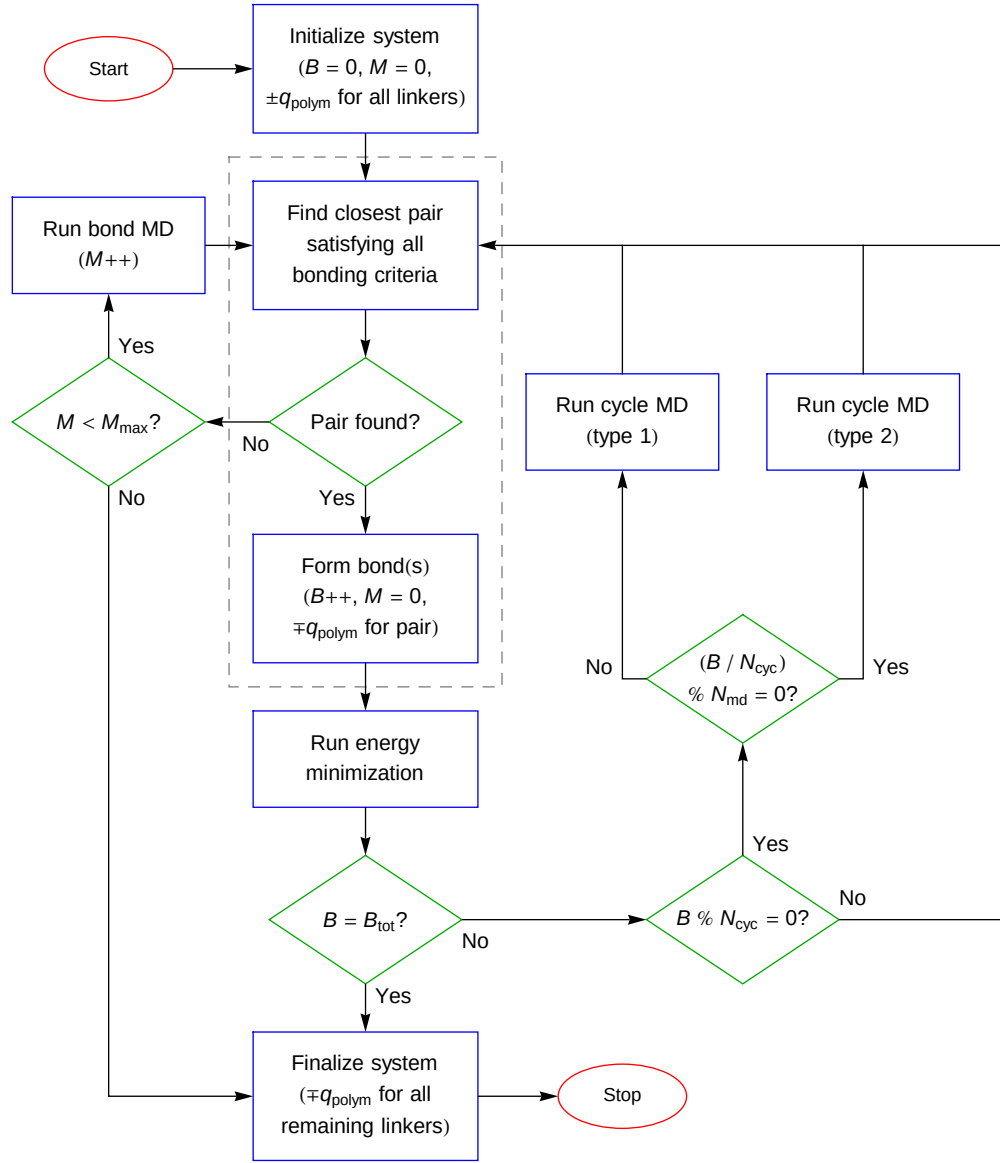


Figure 2. Flowchart of the simulated polymerization algorithm.

The algorithm is implemented using perl scripts to perform the initialization (`polym_init.pl`), a polymerization step (`polym.pl`), and the finalization (`polym_final.pl`), and a python script to control the overall workflow of the polymerization loop (`polym_loop.py`). These files are described in the following sections.

3.2 Polymerization Step

3.2.1 About

The polymerization step identifies the closest pair of linking atoms that satisfies all bonding criteria and updates the connectivity information of the system to reflect the new bond(s) being formed (dashed box in Figure 2). It is carried out by a perl script (`polym.pl`), which is described here.

3.2.2 Syntax

The syntax of the polymerization step code is:

```
./polym.pl -i data.lmps
           -t types.txt
           -s polym.in
           -o new.lmps
```

Below are descriptions of the command-line flags:

1. The `-i` flag indicates the input file of the initial system.
2. The `-t` flag indicates a types file, which identifies the data types in the LAMMPS data file. Its format is described in Section 3.2.3.
3. The `-s` flag indicates the input script, which specifies parameters for the polymerization step. Its format is described in Section 3.2.4.
4. The `-o` flag gives the name of the output file, which contains the system with updated connectivity. This file is only created if a pair of linking atoms is found meeting all bonding criteria.

5. The `-h` flag prints the syntax, then quits the program.

Note: The order in which the flags are specified does not matter.

3.2.3 Types File

The types file (`types.txt`) is used to define the data types in the LAMMPS data file, such that the atom types involved in each bond, angle, dihedral, and improper are indicated. This is necessary to add the proper bonded interactions during the polymerization step after new bonds are formed. The types file acts as a conversion key between the numerical atom types in a LAMMPS data file and more meaningful descriptive strings, such as those given by the force field.

The types file is specified in a text file with a separate section for each data type ('atom types', 'bond types', 'angle types', 'dihedral types', and 'improper types'). In the atom types section, one line is provided for each atom type, where the numerical value from the LAMMPS data file is given first, followed by the associated string. For all other data types, a section is given with a line for each numerical value, along with the atom types involved in that interaction. The atom types for each interaction should correspond to those given in the atom types section, and should be comma separated. The format of the data types file is as follows:

```
atom types
  1 type1
  2 type2
  ...
bond types
  1 type1,type2
  2 type2,type6
  ...
angle types
  1 type1,type2,type4
  2 type3,type2,type6
  ...
```

```

dihedral types
1 type1,type3,type2,type6
2 type2,type6,type4,type3
...
improper types
1 type2,type1,type4,type7
2 type6,type2,type3,type4
...

```

Note: Examples of types files for different polymers are included with the polymerization examples in this distribution (Section 3.6).

3.2.4 Input Script

The input script (`polym.in`) is provided to pass important parameters to the polymerization codes (`polym.pl`, `polym_init.pl`, and `polym_final.pl`). Each definition in the input script is given on a separate line, where the first word of the line is the command specifying which definition is provided. The commands currently implemented in *Polymatic* are listed below. Note that the order of the command lines do not matter, but the order of the arguments for each command do.

Linking atoms: Definition of the linking atoms is a *required* command for the polymerization step. The linking atoms define the atom types between which bonds are to be formed during a polymerization step.

The syntax of the `link` command is:

```
link type1,type1_new type2,type2_new
```

Example:

```
link Lc1,c1 Lc2,c2
```

Here, `type1` and `type2` are the unique atom types of the linking atoms, while `type1_new` and `type2_new` are new atom types assigned to the linking

atoms after they are bonded to prevent them from participating in further polymerization steps. These types should be consistent with the strings defined in the types file, not the numeric values.

Cutoff radius: The cutoff radius is a *required* bonding criterion for the polymerization step. It specifies the maximum distance two linking atoms can be from one another in order to allow bond formation. It should be chosen to ensure that the structure is relaxed adequately during energy minimization and molecular dynamics simulations to follow.

The syntax of the `cutoff` command is:

```
cutoff r
```

Example:

```
cutoff 6.0
```

Artificial charges: Artificial charges can be added to linking atoms to introduce stronger attractions between them during the molecular dynamics simulations. This can improve the efficiency of the polymerization by increasing the chances of two linking atoms coming in contact, as well as to reduce the separation between them when bonds are formed.

The syntax of the `charge` command is:

```
charge q1 q2
```

Example:

```
charge +0.3 -0.3
```

Here, `q1` and `q2` are the charges added to all linking atoms during the polymerization initialization, corresponding to q_{polym} in Figure 2. Their order should be consistent with the linking atoms in the `link` command (i.e., `q1` is added to `type1` and `q2` to `type2`). After a bond is formed between two linking atoms, the charges are subtracted from that atom pair. During the polymerization finalization, artificial charges are subtracted from any remaining linking atoms at the end of the polymerization.

Note that the charges specified in the input script are added to/subtracted from the charges already present for the linking atoms, and do not replace their original charges. Make sure to pay attention to the total charge being added to the system in order to maintain a charge neutral system.

Intramolecular bonds: This bonding criterion is specified to allow or deny bonds to be formed within the same molecule. Intramolecular bonds should be prevented for linear polymers and allowed for network polymers.

The syntax of the `intra` command is:

```
intra  bool N
```

Examples:

```
intra  false
intra  true 5
```

Here, `bool` is a Boolean indicating if intramolecular bonding should be allowed (true) or prevented (false). If true, `N` (1 to 5) indicates the number of bonds separating two linking atoms for which they will not be bonded. Note that the default value for intramolecular bonding is false.

Atom connectivity: The remaining commands listed in this section involve atoms within the structure other than the linking atoms. In order to identify those atoms, an atom definition needs to be provided. This is done using a description of the atom connectivity from the linking atoms to other atoms involved and their corresponding atom types (see below).

The syntax of the `connect` command is:

```
connect  N
1 connected_atoms_1
2 connected_atoms_2
...
```

An example of the atom definition is provided here for ladder polymer PIM-1, which will be used to illustrate the remaining commands listed in this section.

Two nearby repeat units of PIM-1 are shown in Figure 3a, where the linking atoms are marked by blue and read beads (atoms 1 and 2, respectively) and all other identified atoms (atoms 3–12) are marked by green beads. The connectivity for this example is defined as:

```
connect 7
1 3,4
2 5
3 6
4 7
5 8,9
8 10,11
9 12
```

Here, N is the number of atoms for which a connectivity definition is provided. The connectivity for each atom is then given on a separate line, with the atom number followed by a comma-separated list of the atoms to which it is bonded. Only the connectivity to atoms referenced in other commands is required.

Note that these atom numbers are defined for use in the polymerization steps only, and are not the same as those in the LAMMPS data file. The linking atoms are assumed to be atoms 1 and 2 in this definition, in the same order they were defined in the `link` command. The connectivity definition of all remaining atoms should be unique.

Atom types: The atom definition required for the remaining commands listed in this section also requires the identification of the atom types, in addition to the atom connectivity from the linking atoms to other atoms involved (see above). These are the atom types corresponding to the string values in the types file.

The syntax of the `types` command is:

```
types N
1 type1
2 type2
...
```

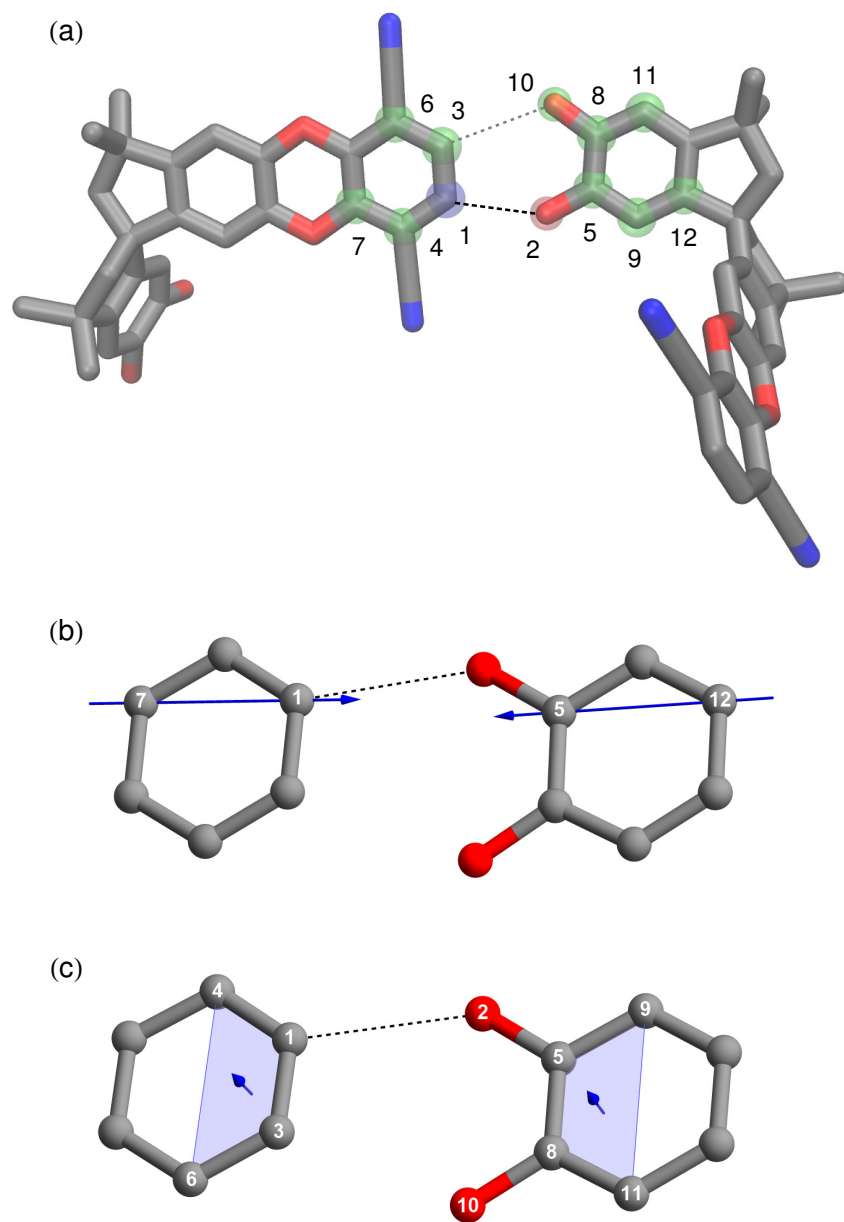


Figure 3. Illustration of polymerization step definitions for PIM-1.

For the PIM-1 example discussed above and illustrated in Figure 3a, the types section is given as:

```
types 12
1 LCSO
2 LOAO
3 LCSO
4 CAO
5 CSO
6 CAO
7 CSO
8 CSO
9 CA1
10 LOAO
11 CA1
12 CAO
```

Here, N is the number of atoms in the atom definition. The atom number and atom type for each atom are then given on a separate line. The atom types should correspond to the string values in the types file.

Extra bonds: Extra bonds can be defined for more complicated systems, which are formed in addition to the primary bond between the linking atoms.

The syntax of the **bond** command is:

```
bond atom1 atom2
```

For the PIM-1 example discussed above and illustrated in Figure 3a, an additional bond is needed to form the ladder backbone (marked in the figure by the gray dashed line). The extra bond for this example is given as:

```
bond 3 10
```

Here, **atom1** and **atom2** correspond to the atom numbers of the atom definition provided by the **connect** and **types** commands. If their atom types are consistent with the linking atom types (in the order specified by the **link** command), then their atom types and charges are changed after bond formation in accordance with the **link** and **charge** commands.

Alignment checks: Alignment checks impose conditions on angles between vectors or normal vectors to planes to ensure that reasonable bonds are formed during a polymerization step. Multiple checks can be defined, each of which should be given as its own command.

The syntax of the `align` command is:

```
align atoms1 atoms2 conditions
```

For the PIM-1 example discussed above and illustrated in Figure 3a, an angle between vectors as shown in Figure 3b must satisfy $\theta > 135^\circ$, and an angle between normal vectors to planes as shown in Figure 3c must satisfy $\theta < 40^\circ$ or $\theta > 140^\circ$. This example is given as:

```
align 1,7 5,12 >135
align 4,1,3,6 9,5,8,11 <40,>140,||
```

Here, `atoms1` and `atoms2` are comma-separated lists of atoms making up the vectors or planes, where the atom numbers reference the atom definition provided by the `connect` and `types` commands. A vector is defined by exactly two atoms, its head and tail in that order (e.g., `head,tail`), while a plane is defined by at least three atoms.

The `conditions` for the angle should be specified by strict inequalities (i.e., `<` or `>`). It is one item if only one condition is necessary (e.g., `cond`). To impose two conditions, they should be given comma separated with an AND (`&`) or OR (`||`) logical operator last (e.g., `cond1,cond2,operator`).

3.2.5 Output

The primary output of the polymerization step code is a LAMMPS data file containing the system with updated connectivity (`new.lmps`). In addition, a line is written to standard output for each bond added (i.e., for the closest pair of linking atoms and any extra bonds), specifying the atoms involved (numbers corresponding to the data file) and their separation distance (in angstroms). For example:

```
Pair: 2.73 A (29,61)
Extra bond: 3.20 A (28,59)
```

Note: A new LAMMPS data file (`new.lmps`) is written only if the polymerization step is successful. If no linking atoms are found within the system meeting all bonding criteria, no new file is written and an incomplete status code (3) is returned.

3.3 Initialization and Finalization

3.3.1 About

The initialization and finalization of the system are performed at the beginning and end of the polymerization loop, respectively, as indicated in the flowchart in Figure 2. Currently, the only task included in the initialization is the addition of artificial charges to all linking atoms within the system, if specified by a `charge` command in the polymerization input script (Section 3.2.4). Likewise, in the finalization script (`polym_final.pl`), the artificial charges are subtracted from any remaining linking atoms in the system. The finalization script also unwraps the coordinates of each molecule so that no bonds span the periodic boundaries for better visualization.

The initialization and finalization steps do not have to be included in the simulated polymerization algorithm if these tasks are not needed. Furthermore, these scripts could be updated to perform other tasks desired for initialization or finalization of the simulation, such as changing the atom types of all remaining linking atoms at the end of the simulation.

3.3.2 Syntax

The syntax of the polymerization initialization and finalization codes is:

```
./polym_*.pl -i data.lmps
              -t types.txt
              -s polym.in
              -o new.lmps
```

Here, `polym_*.pl` should be either `polym_init.pl` or `polym_final.pl`.

Below are descriptions of the command-line flags:

1. The `-i` flag indicates the input file of the initial system.
2. The `-t` flag indicates a types file, which identifies the data types in the LAMMPS data file. Its format is described in Section 3.2.3.
3. The `-s` flag indicates the input script, which specifies parameters for the polymerization step. Its format is described in Section 3.2.4.
4. The `-o` flag gives the name of the output file, which contains the updated system.
5. The `-h` flag prints the syntax, then quits the program.

Note: The order in which the flags are specified does not matter.

3.3.3 Output

The only output of the initialization and finalization scripts is a LAMMPS data file with the updated system. Currently, this includes the addition or subtraction of artificial charges to all linking atoms within the system if the **charge** command is defined in the input script (`polym.in`). The finalization script also adjusts the coordinates to unwrap each molecule for improved visualization of the polymers.

3.4 Polymerization Loop

3.4.1 About

The polymerization loop python script (`polym_loop.py`) controls the workflow of the entire simulated polymerization algorithm, performing the appropriate tasks as illustrated in Figure 2. It is responsible for (i) calling LAMMPS to perform energy minimization and molecular dynamics simulations, (ii) calling the perl scripts described above to perform a polymerization step, initialization, or finalization, and (iii) handling file organization and maintenance.

3.4.2 Syntax

The syntax of the polymerization loop script is:

```
./polym_loop.py
```

Note: This script takes no command-line arguments. Instead, all parameters are defined in the script or in the LAMMPS input scripts.

3.4.3 Parameters and Input Scripts

The polymerization loop script defines the following parameters related to the workflow of the algorithm:

1. **bonds**, the initial number of bonds formed in the polymerization, which corresponds to B in Figure 2. This variable should be 0, unless restarting the polymerization from the middle.
2. **bonds_tot**, the total number of bonds to be formed in the polymerization, which corresponds to B_{tot} in Figure 2. The polymerization is stopped when $B = B_{\text{tot}}$.
3. **bonds_cyc**, the number of bonds to form in each cycle, which corresponds to N_{cyc} in Figure 2. A cycle MD is performed when the logical statement $B \bmod N_{\text{cyc}} = 0$ is true.
4. **md_cyc**, how often (in number of cycles) a cycle MD should be of type 2, which corresponds to N_{md} in Figure 2. A type 2 cycle MD is performed when the logical statement $(B/N_{\text{cyc}}) \bmod N_{\text{md}} = 0$ is true.
5. **md_max**, the maximum number of MD steps to perform during a bond attempt, which corresponds to M_{max} in Figure 2. The polymerization is quit if $M = M_{\text{max}}$.
6. **keep**, indicates whether all interim files created during the polymerization should be kept (1) or deleted (0). If deleted, only the initial and final structures are provided.

The polymerization loop script defines the file names of the following input files needed during the polymerization:

1. `input_polym`, the input file for the polymerization scripts, which is described in Section 3.2.4.
2. `input_min`, the LAMMPS input script for the energy minimization step, which is performed after a successful polymerization step, as indicated in Figure 2.
3. `input_md0`, the LAMMPS input script for the bond molecular dynamics step, which is performed if no bond meeting all criteria is found, as indicated in Figure 2.
4. `input_md1`, the LAMMPS input script for the cycle type 1 molecular dynamics step, which is performed between polymerization cycles, as indicated in Figure 2.
5. `input_md2`, the LAMMPS input script for the cycle type 2 molecular dynamics step, which is performed between polymerization cycles, as indicated in Figure 2.

The polymerization loop script defines the file names of the following polymerization scripts:

1. `script_step`, the polymerization step script, as described in Section 3.2. This is `polym.pl` in the *Polymatic* distribution.
2. `script_init`, the polymerization initialization script, as described in Section 3.3. This is `polym_init.pl` in the *Polymatic* distribution. If set to 0, no initialization is performed.
3. `script_final`, the polymerization finalization script, as described in Section 3.3. This is `polym_final.pl` in the *Polymatic* distribution. If set to 0, no finalization is performed.

Lastly, the polymerization loop script defines the command-line execution of LAMMPS as the `lmps` parameter. For example:

```
lmps = 'mpirun -np 8 /file/path/to/lammps-*/src/lmp_mpi'
```

3.4.4 Setup and Output

The polymerization loop script was written assuming a certain file architecture. The base directory should contain the LAMMPS data file of the initial system and the corresponding types file. There should also be a `scripts` directory that contains the input files and polymerization scripts described in Section 3.4.3. The initial setup should be as follows:

```
polym_loop.py      # Polymerization loop script
data.lmps          # LAMMPS data file of initial system
types.txt          # Types file
scripts/
  md0.in           # LAMMPS input script, bond MD
  md1.in           # LAMMPS input script, cycle MD type 1
  md2.in           # LAMMPS input script, cycle MD type 2
  min.in           # LAMMPS input script, EM
  polym.in         # Polymatic input script
  polym.pl         # Polymerization step script
  polym_init.pl    # Polymerization initialization script
  polym_final.pl   # Polymerization finalization script
```

The main output of the polymerization loop is a LAMMPS data file containing the final polymerized system, which is named `final.lmps`. Progress of the polymerization is also written to standard output, including a header and footer listing the polymerization parameters and completion status. A few lines are also written for each bonding step, which includes the output generated by the polymerization step, as described in Section 3.2.5, as well as the number of bond attempts required for that step, defined by M in Figure 2. For example:

```
Step 003:
  Pair: 4.03 A (503,136)
  Attempts: 1
```

Throughout the polymerization, new files and directories are created while polymerization steps and simulations are performed. Intermediate files can be kept or deleted using the `keep` parameter (Section 3.4.3). These files are described here.

The most up-to-date system is always stored as a LAMMPS data file named `temp.lmps` in the base directory. It is updated after each successful polymerization step and cycle MD simulation.

A directory is created for each step, named `step_N`, corresponding to B in Figure 2. It contains three LAMMPS data files: (i) the initial system, `init.lmps`, which is copied from `temp.lmps` in the base directory, (ii) the updated system after a successful polymerization step, `data.lmps`, and (iii) the system after an energy minimization, `min.lmps`, which then replaces `temp.lmps` in the base directory. For example:

```
step_001/
  init.lmps
  data.lmps
  min.lmps
```

A directory is created for each bond MD step, named `md_M`, corresponding to M in Figure 2. It contains two LAMMPS data files: (i) the initial system, `data.lmps`, which is copied from `init.lmps` in the bond step directory, and (ii) the system after the MD step, `md.lmps`, which then replaces `init.lmps` in the bond step directory. For example:

```
step_012/
  init.lmps
  data.lmps
  min.lmps
  md_001/
    data.lmps
    md.lmps
```

A directory is created for each cycle MD step, named `step_N_md`, corresponding to B in Figure 2. It contains two LAMMPS data files: (i) the initial system, `data.lmps`, which is copied from `temp.lmps` in the base directory, and (ii) the system after the MD step, `md.lmps`, which then replaces `temp.lmps` in the base directory. For example:

```
step_005_md/
  data.lmps
  md.lmps
```

Note: Output files generated during LAMMPS simulations are also written to their respective directories, but are not discussed here.

3.5 Usage

Below is a list of notes about the polymerization codes that may be useful for using and/or modifying them:

1. Some subroutines in the `Polymatic.pm` module are used by the polymerization scripts. It therefore needs to be included in the same directory or a path to its directory should be defined (e.g., with `use lib`). The polymerization loop script, `polym_loop.py`, assumes that each script defines the location of the module with `use lib`. These file paths should be modified accordingly to match your setup.
2. It is important that all force field coefficients needed during the polymerization are provided in the original data file. This includes any new data types (atom, bond, angle, dihedral, and improper) that are added to the system during a polymerization step. The `polym.pl` script will throw an error if a necessary data type is missing.
3. Currently, `polym_init.pl` and `polym_final.pl` are only necessary to add/remove artificial charges and unwrap the molecules for better visualization. If these tasks are not needed, these scripts can be omitted by setting their file names to 0 in `polym_loop.py` (Section 3.4.3).
4. It is possible to restart a polymerization from any bond step. If the polymerization loop failed due to reaching the maximum number of bond MD steps, then the final system should be written to `final.lmps`. If it fails in some other way, `temp.lmps` will hold the most up-to-date system. Note that `temp.lmps` still has artificial charges added to linking atoms (if artificial charges were used), which should be removed by running `polym_final.pl` before restarting the polymerization. To restart from the last finished bond, copy the final system, `final.lmps`, to `data.lmps` and change the `bond` parameter in `polym_loop.py` accordingly (Section 3.4.3). Note that, if intermediate files are being kept, the directory of the failed step needs to be removed before restarting.

5. Currently, `polym.pl` works only for cubic simulation boxes with periodic boundary conditions in all dimensions. Updates are required if other system setups are desired.
6. In the polymerization scripts, the molecular system is stored in the `%sys` hash. The parameters from the polymerization input script are stored in the `%inp` hash.
7. In `polym.pl`, the identification of the closest pair of linking atoms in the system is performed in the `findPair` subroutine. From there, all bonding criteria are checked for each pair, some of which are performed in a separate subroutine (i.e., `alignment`). The selected pair is stored in the `@pair` array. Any new bonding criteria implemented in *Polymatic*, should be included here.
8. In `polym.pl`, all updates to the system information and connectivity are made in the `makeUpdates` subroutine. When a new bond is added, all possible angles, dihedrals, and impropers resulting from the new bonding are also automatically added to the system definition. Note that, if the number of angle, dihedral, or improper types is 0, no angles, dihedrals, or impropers are added, respectively.
9. The polymerization input script is read in the `readPolymInput` subroutine of the `Polymatic.pm` module. If new commands for the input script are implemented, they should be added here.
10. In `polym_loop.py`, the `polym_loop` subroutine handles the workflow of the *Polymatic* algorithm (Figure 2). The polymerization initialization and finalization scripts are run in the `polym_init` and `polym_final` subroutines. The `polym_step` subroutine handles a single polymerization step, including running the polymerization step script and any necessary bond MD steps. Energy minimization and MD steps are run in the `em` and `md` subroutines, respectively.
11. Files are read and written in the LAMMPS data format with calls to the `readLammps` and `writeLammps` subroutines in the `Polymatic.pm` module. This would be the place to start in order to adapt the code to work with other file types (such as those included in the `Polymatic.pm` module).

Note: The *Polymatic* distribution uses hashes to store information defining a molecular system and polymerization parameters, which are discussed in Appendix A. In addition, the `Polymatic.pm` module can read and write several popular file types to and from molecular system hashes, as described in Appendix B. This information may be useful if modifying the codes.

3.6 Examples

The input and output files for each example are provided with the *Polymatic* distribution in the corresponding `Example*` directory.

Example 4: This example performs a single polymerization step on two nearby polystyrene repeat units, which are illustrated in Figure 4. The original system is defined in the data file, `data.lmps`, and types file, `types.txt`. The polymerization input file, `polym.in`, specifies the linking atom types ('Lc1' and 'Lc1') and a cutoff radius of 6 Å.

```
./polym.pl -i data.lmps -t types.txt -s polym.in -o new.lmps
```

In Figure 4, the closest pair of linking atoms is indicated by the blue and red beads connected by a dashed line. On successful completion of the polymerization step, a line is printed to standard output that specifies that a bond was made between atoms 23 and 8 at a separation distance of 3.20 Å:

```
Pair: 3.20 A (23,8)
```

The atom types of the bonded linking atoms were updated to 'c1' and 'c2', which is reflected in the updated system, `new.lmps`. Comparison of the original and new files summarizes the new connectivity added during the polymerization step:

# data.lmps	# new.lmps
32 atoms	32 atoms
32 bonds	33 bonds
48 angles	54 angles
64 dihedrals	79 dihedrals
16 impropers	22 impropers

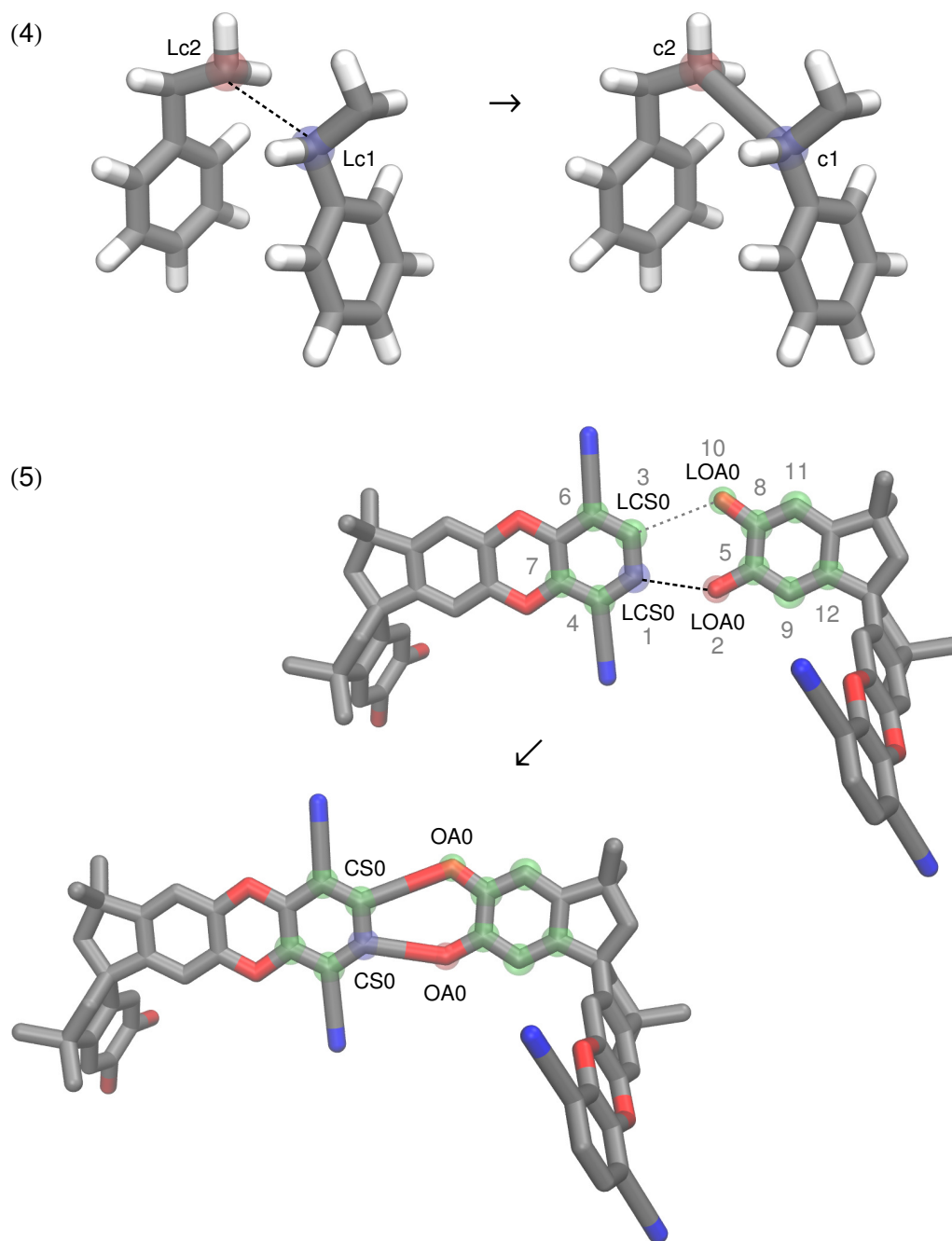


Figure 4. Illustrations of Examples 4 and 5.

Example 5: This example performs a single polymerization step on two nearby PIM-1 repeat units, which are illustrated in Figure 4. The original system is defined in the data file, `data.lmps`, and types file, `types.txt`. The polymerization input file, `polym.in`, specifies the linking atom types ('LCS0' and 'LOA0'), artificial charges of $\pm 0.3 e$, a cutoff radius of 6 Å, an additional bond, two alignment checks, and the connectivity definition.

```
./polym.pl -i data.lmps -t types.txt -s polym.in -o new.lmps
```

In Figure 4, the closest pair of linking atoms is indicated by the blue and red beads, while other atoms defined in the input script are indicated by the green beads. The bond between the closest pair of linking atoms is indicated by the black dashed line, the extra bond by the gray dashed line.

On successful completion of the polymerization step, two lines are printed to standard output that specify that a bond was made between atoms 29 and 61 at a separation distance of 2.73 Å, and an extra bond between atoms 28 and 59 at a separation distance of 3.20 Å:

```
Pair: 2.73 A (29,61)
Extra bond: 3.20 A (28,59)
```

The atom types of the bonded linking atoms (both the closest pair and the pair in the extra bond) were updated to 'CS0' and 'OA0', and artificial charges were removed from both pairs, which is reflected in the updated system, `new.lmps`. Comparison of the original and new files summarizes the new connectivity added during the polymerization step:

# data.lmps	# new.lmps
70 atoms	70 atoms
80 bonds	82 bonds
132 angles	138 angles
194 dihedrals	209 dihedrals

Note that a data file, `data_bad.lmps`, is also provided for a pair of PIM-1 monomers for which the alignment checks fail. If the example is run again with this system, the polymerization step does not complete successfully and so no new system is generated.

Example 6: This example performs the entire simulated polymerization algorithm on a periodic system of 50 polystyrene repeat units, which is illustrated in Figure 5a. The packed system is defined in the data file, `data.lmps`, with the types file, `types.txt`, and polymerization input file, `polym.in` (the same as in Example 4). All necessary input files and scripts are given in the `scripts` directory.

The parameters of the polymerization are specified in `polym_loop.py`, which indicate that 49 total bonds are to be formed (`bonds_tot=49`) with a maximum of 20 bond attempts per step (`md_max=20`). Cycles are made up of 5 bonds (`bonds_cyc=5`), between which cycle MD steps are performed, every third of type 2 (`md_cyc=3`). The first MD type is a shorter NVT simulation at 600 K, the second a longer NPT simulation at 300 K.

```
./polym_loop.py
```

The final polymerized system is written to the data file, `final.lmps`, which is illustrated in Figure 5b with the polymer backbone colored blue for clarity. In addition, output is written to standard output summarizing the polymerization results. This includes the parameters, the output for each polymerization step (as discussed in Section 3.2.5), and a summary of the run. In this example, 49 of 49 bonds were completed, resulting in 100% completion.

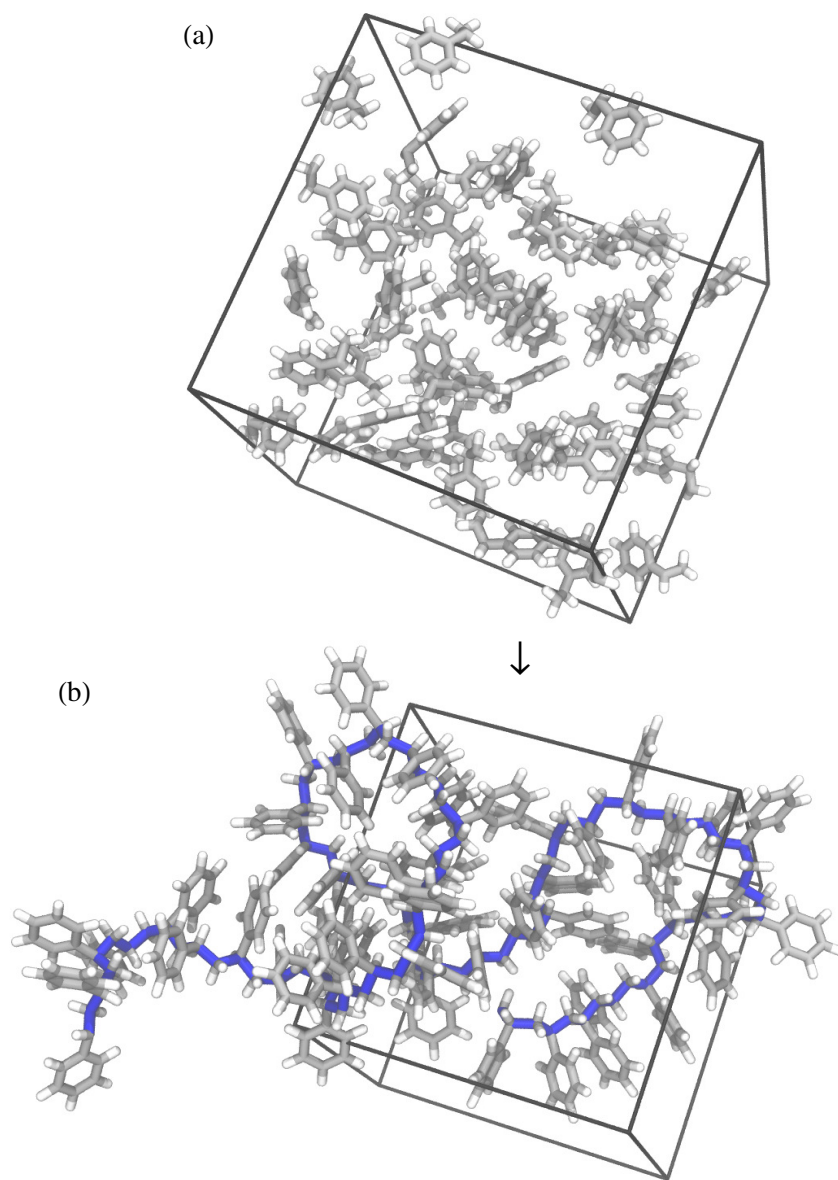


Figure 5. Illustration of Example 6.

A Data Structures

A.1 About

Within the *Polymatic* codes, nested data structures of hashes, arrays, and scalars are used to store molecular systems and input script parameters. In the following sections, hashes are denoted by `% = { 'key' => value }` with a key/value pair for each entry. The key is string, while the value can be a reference to another data structure or a scalar. Arrays are denoted by `@ = [ val_i ]`, where values are stored as elements in an indexed list starting at position 0. Again, the values can be a reference to another data structure or a scalar. Scalars (numbers and strings) are denoted by `$`.

A.2 Molecular System

A single molecular system is stored as a hash with the following layout:

```
%sys =  
{  
  'header' => $str,  
  'flags' => %flags,  
  'boxDims' => %boxDims,  
  'neighList' => %neighList,  
  'mols' => %mols,  
  'atoms' => %atoms,  
  'bonds' => %bonds,  
  'angles' => %angles,  
  'diheds' => %diheds,  
  'improps' => %improps,  
  'atomTypes' => %atomTypes,  
  'bondTypes' => %bondTypes,  
  'angleTypes' => %angleTypes,  
  'dihedTypes' => %dihedTypes,  
  'impropTypes' => %impropTypes  
}
```

The `header` key points to a string title (`$str`) for the molecular system. All other keys are described in the following sections.

A.2.1 Flags

The `flags` key points to a hash storing any flags defined for the molecular system. These are used to indicate certain properties of the system that may be important for conversion to different file types, for instance.

The layout of the flags hash is:

```
%flags =  
{  
  'ang' => bool  
}
```

Here, one key is given for each flag with a Boolean value to indicate if that flag is true or false. Currently, only the `ang` key is utilized, which indicates if the coordinates in the molecular system are in angstroms.

To access the Boolean of the `ang` flag, for example:

```
$sys{'flags'}{'ang'}
```

A.2.2 Periodic Box

The `boxDims` key points to a hash storing the dimensions of the periodic box for the molecular system.

The layout of the periodic box hash is:

```
%boxDims =  
{  
  'x' => [$lo, $hi, $len],  
  'y' => [$lo, $hi, $len],  
  'z' => [$lo, $hi, $len]  
}
```

Here, one key is given for each dimension, each of which points to an array with three values: `$lo` and `$hi` are the minimum and maximum values of the box on that axis, respectively, and `$len` is the total length of the box in that dimension (i.e., `$len = $hi - $lo`).

To access the length of the box in the x -direction, for example:

```
$sys{'boxDims'}{'x'}[2]
```

A.2.3 Neighbor List

The `neighList` key points to a hash storing information about the neighbor list for the molecular system. Currently, neighbor lists are implemented in the *Polymatic* packing code for cubic boxes only.

The layout of the neighbor list hash is:

```
%neighList =
{
  'num' => $num,
  'width' => $width,
  'cells' => @cells
}
```

Here, `$num` is the number of cells in each direction and `$width` is the width of each cell. The `cells` key points to a three-dimensional array (`@cells`) storing the atoms in each cell of the neighbor list. Each element of the three-dimensional array is an array of the atoms in the given cell, where the atoms are referenced by their atom numbers.

To access the array of atoms in cell (u,v,w), for example:

```
@{$sys{'neighList'}{'cells'}[$u][$v][$w]}
```

A.2.4 Molecules

The `mols` key points to a hash storing information about the molecules in the molecular system.

The layout of the molecules hash is:

```
%mols =
{
  'count' => $num,
  'atoms' => [0, @atoms1, @atoms2, ... ]
}
```

Here, `$num` is the number of molecules in the molecular system. The `atoms` key points to an array storing an array of atoms for each molecule (e.g., `@atoms1` corresponds to molecule 1), where the atoms are referenced by their atom numbers. Note that the first element of the array is 0 so that the indexing for the molecule atoms starts at 1.

To access the array of atoms in the second molecule, for example:

```
@{$sys{'mols'}{'atoms'}[2]}
```

A.2.5 Atoms

The `atoms` key points to a hash storing information about the atoms in the molecular system.

The layout of the atoms hash is:

```
%atoms =
{
  'count' => $num,
  'mol' => [0, $mol1, $mol2, ... ],
  'type' => [0, $type1, $type2, ... ],
  'q' => [0, $q1, $q2, ... ],
  'pos' => [0, @pos1, @pos2, ... ],
  'bonded' => [0, @bonded1, @bonded2, ... ],
  'bonds' => [0, @bonds1, @bonds2, ... ],
  'angles' => [0, @angles1, @angles2, ... ],
  'diheds' => [0, @diheds1, @diheds2, ... ],
  'improps' => [0, @improps1, @improps2, ... ]
}
```

Here, `$num` is the number of atoms in the molecular system. The remaining keys point to arrays storing data about each atom. For each array, the first element is 0 so that the indexing of the data starts at 1 to correspond with the atom numbers.

The arrays paired with the `mol`, `type`, and `q` keys contain scalars of the molecule number, numerical atom type, and charge for each atom (e.g., `$mol1`, `$type1`, and `$q1` correspond to atom 1). The `pos` key lists the positions for each atom as an array (e.g., `@pos1 = [$x1, $y1, $z1]`, corresponding to atom 1).

The `bonded` key contains arrays of the atoms bonded to each atom (e.g., `@bonded1` corresponds to atom 1), where the atoms are referenced by their atom numbers. The arrays paired with the `bonds`, `angles`, `diheds`, and `improps` keys contain arrays of the bonds, angles, dihedrals, and impropers in which each atom is involved (e.g., `@bonds1`, `@angles1`, `@diheds1`, and `@improps1` correspond to atom 1). Here, the bonds are referenced by the bond number, the angles by the angle number, etc.

To access the array containing the position of atom 16, for example:

```
@{$sys{'atoms'}}{'pos'}[16]}
```

A.2.6 Bonded Interactions

The `bonds`, `angles`, `diheds`, and `improps` keys point to hashes storing information about the bonded interactions in the molecular system.

The layout of the interaction hashes is:

```
%int =
{
  'count' => $num,
  'type' => [0, $type1, $type2, ... ],
  'atoms' => [0, @atoms1, @atoms2, ... ]
}
```

Note that the `%bonds`, `%angles`, `%diheds`, and `%improps` hashes all follow this generic `%int` format.

Here, `$num` is the number of interactions (bonds, angle, dihedrals, or impropers) in the molecular system. The `types` key points to an array containing scalars of the numerical data types for each interaction (e.g., `$type1` corresponds to interaction 1). The `atoms` key points to arrays of the atoms involved in each interactions (e.g., `@atoms1` corresponds to interaction 1). The atom arrays in these cases have two atoms listed for bonds, three atoms for angles, and four atoms for dihedrals and impropers (e.g., `@atoms1 = [$a1, $a2, $a3]`, corresponding to the atoms in angle 1).

Note that the first elements of the arrays (corresponding to the `types` and `atoms` keys) are 0 so that the indexing of the data starts at 1 to correspond with the interaction numbers.

To access the array containing the atoms in dihedral 4, for example:

```
@{$sys{'diheds'}{'atoms'}[4]}
```

A.2.7 Data Types

The `atomTypes`, `bondTypes`, `angleTypes`, `dihedTypes`, and `impropTypes` keys point to hashes storing information about the data types in the molecular system.

The layout of the data types hashes is:

```
%types =
{
  'count' => $num,
  'name' => [0, $str1, $str2, ... ],
  'num' => {$str1 => $num1, $str2 => $num2, ... },
  'coeffs' => %coeffs
}
```

Note that the `%atomTypes`, `%bondTypes`, `%angleTypes`, `%dihedTypes`, and `%impropTypes` hashes all follow this generic `%types` format.

Here, `$num` is the number of data types (atom, bond, angle, dihedral, or improper) in the molecular system. The `name` key points to an array listing the string values corresponding to the numerical data types (e.g., `$str1`

corresponds to type 1). The `num` key points to a hash where the keys are the string data types and the values are the numerical data types (e.g., `$str1` and `$num1` correspond to type 1).

The `coeffs` key points to a hash containing force field coefficient information for the data types. Note that the information stored in the hash is dependent on which data type is being considered (atom, bond, angle, dihedral, or improper) and on the force field functional form (e.g., class I or II). The setup of the coefficients hashes here is designed to be consistent with the nomenclature and format of the LAMMPS data file.

For atom types, the layout of the coefficients hash is:

```
%coeffs =  
{  
  'mass' => [0, $mass1, $mass2, ... ],  
  'pair' => [0, @coeffs1, @coeffs2, ... ]  
}
```

Here, the `mass` and `pair` keys define the masses and pair coefficients for each atom type (e.g., `$mass1` and `@coeffs1` correspond to atom type 1).

For bond types, the layout of the coefficients hash is:

```
%coeffs =  
{  
  'bond' => [0, @coeffs1, @coeffs2, ... ]  
}
```

Here, the `bond` key defines the bond coefficients for each bond type (e.g., `@coeffs1` corresponds to bond type 1).

For angle types, the layout of the coefficients hash is:

```
%coeffs =  
{  
  'angle' => [0, @coeffs1, @coeffs2, ... ],  
  'bondBond' => [0, @coeffs1, @coeffs2, ... ],  
  'bondAng' => [0, @coeffs1, @coeffs2, ... ]  
}
```

Here, the **angle**, **bondBond**, and **bondAng** keys define the angle, bond-bond, and bond-angle coefficients for each angle type (e.g., **@coeffs1** corresponds to angle type 1). The bond-bond and bond-angle terms are part of a class II force field definition.

For dihedral types, the layout of the coefficients hash is:

```
%coeffs =
{
  'dihed' => [0, @coeffs1, @coeffs2, ... ],
  'midBondTors' => [0, @coeffs1, @coeffs2, ... ],
  'endBondTors' => [0, @coeffs1, @coeffs2, ... ],
  'angTors' => [0, @coeffs1, @coeffs2, ... ],
  'angAngTors' => [0, @coeffs1, @coeffs2, ... ],
  'bondBond13' => [0, @coeffs1, @coeffs2, ... ]
}
```

Here, the **dihed**, **midBondTors**, **endBondTors**, **angTors**, **angAngTors**, and **bondBond13** keys define the dihedral, middle-bond-torsion, end-bond-torsion, angle-torsion, angle-angle-torsion, and bond-bond-13 coefficients for each dihedral type (e.g., **@coeffs1** corresponds to dihedral type 1). The middle-bond-torsion, end-bond-torsion, angle-torsion, angle-angle-torsion, and bond-bond-13 terms are part of a class II force field definition.

For improper types, the layout of the coefficients hash is:

```
%coeffs =
{
  'improp' => [0, @coeffs1, @coeffs2, ... ],
  'angAng' => [0, @coeffs1, @coeffs2, ... ]
}
```

Here, the **improp** and **angAng** keys define the improper and angle-angle coefficients for each improper type (e.g., **@coeffs1** corresponds to improper type 1). The angle-angle terms are part of a class II force field definition.

Note that for the array corresponding to each key in all coefficients hashes, the first element is 0 so that the indexing of the data starts at 1 to correspond with the data types numbers.

To access the string value associated with bond type 2, for example:

```
$sys{'bondTypes'}{'name'}[2]
```

To access the array of angle coefficients for angle type 5, for example:

```
@{$sys{'angleTypes'}{'coeffs'}{'angle'}[5]}
```

A.3 Input Script Parameters

The polymerization input script parameters (Section 3.2.4) are stored as a hash with the following layout:

```
%inp =  
{  
  'link' => @link,  
  'cutoff' => $cutoff,  
  'charge' => @charge,  
  'intra' => $num,  
  'connect' => @connect,  
  'types' => @types,  
  'bond' => @bonds,  
  'align' => %align  
}
```

The `cutoff` key gives the cutoff distance (`$cutoff`) defined by the `cutoff` command in the polymerization input script. The `intra` key gives the `intra` command argument, which is the number of bonds between atoms for which intramolecular bonds should be prevented (`$num`). It is defined only if the `intra` command is set as true in the polymerization input script. All other keys are described in the following sections.

A.3.1 Link Command

The `link` key points to an array storing the atom types defined by the `link` command in the polymerization input script.

The layout of the link array is:

```
@link = [[ $type1, $type1n ], [ $type2, $type2n ]]
```

Here, `$type1` and `$type2` are the string data types for the linking atoms in the polymerization steps, while `$type1n` and `$type2n` are the new string data types assigned to linking atoms after a bond is formed between them during a polymerization step.

To access the string atom type of the first linking atom, for example:

```
$inp{'link'}[0][0]
```

A.3.2 Charge Command

The `charge` key points to an array storing the artificial charges associated with the linking atoms defined by the `charge` command in the polymerization input script.

The layout of the charge array is:

```
@charge = [ $q1, $q2 ]
```

Here, `$q1` and `$q2` are the artificial charges assigned to linking atoms in the polymerization scripts.

To access the artificial charge of the second linking atom, for example:

```
$inp{'charge'}[1]
```

A.3.3 Connect Command

The `connect` key points to an array storing all atom connections for the atom definition as indicated by the `connect` command in the polymerization input script.

The layout of the connect array is:

```
@connect = [ 0, @bonded1, @bonded2, ... ]
```

Here, an element of the array is defined for each atom given in the **connect** command, which holds an array of all atoms bonded to that atom (e.g., **@bonded1** corresponds to atom 1), where the atoms are referenced by their atom numbers defined in the input script. Note that some array elements will be left empty if no connection definition is provided for that atom in the **connect** command.

To access the array of atoms bonded to atom 6, for example:

```
@{$inp{'connect'}[6]}
```

A.3.4 Types Command

The **types** key points to an array storing all string atom types for the atom definition as indicated by the **types** command in the polymerization input script.

The layout of the connect array is:

```
@types = [0, $type1, $type2, ... ]
```

Here, the array includes the string atom type of each atom given in the **types** command (e.g., **\$type1** corresponds to atoms 1), where the atoms are referenced by their atom numbers defined in the input script.

To access the string atom type of atom 4, for example:

```
$inp{'types'}[4]
```

A.3.5 Bond Command

The **bond** key points to an array storing all extra bonds to be formed during a polymerization step, as indicated by the **bond** command in the polymerization input script.

The layout of the connect array is:

```
@bonds = [@pair1, @pair2, ... ]
```

Here, each array element indicates the atom pairs involved in an extra bond (e.g., `@pair1 = [$a1, $a2]`, corresponding to the first extra bond), where the atoms are referenced by their atom numbers defined in the input script.

To access the array of atoms in the first extra bond, for example:

```
@{$inp{'bond'}[0]}
```

A.3.6 Align Command

The `align` key points to a hash storing information about all alignment checks to be performed during a polymerization step, as indicated by the `align` command in the polymerization input script.

The layout of the alignment hash is:

```
%align =
{
  'atoms' => [@atoms1, @atoms2, ... ],
  'cond' => [@cond1, @cond2, ... ]
}
```

Here, the `atoms` key gives an array of arrays, one for each alignment check. The array for a single alignment check defines the atoms in both groups involved (e.g., `@atoms1 = [@group1, @group2]`, corresponding to the first alignment check). Note that these atoms are referenced by their atom numbers defined in the input script.

The `cond` key gives a corresponding array of arrays, one for each alignment check. An array for a single alignment check defines the angle conditions, which should be either one or three elements (e.g., `@cond1 = [$c1]` or `@cond1 = [$c1, $c2, $op]`, corresponding to the conditions of the first alignment check).

To access the array containing the second group of atoms of the first alignment check, for example:

```
@{$inp{'align'}{'atoms'}[0][1]}
```

B File Conversion

B.1 About

The `Polymatic.pm` module included in this distribution contains subroutines for reading and writing different file types for molecular systems. It can also be used to determine the connectivity of a system (angles, dihedrals, impropers, and molecules) given that bonds are defined. These can be used for file conversion purposes to (i) visualize the molecular systems in programs like [VMD](#) and (ii) extend the *Polymatic* codes for other file types and software packages.

B.2 Reading and Writing Files

In the `Polymatic.pm` module, subroutines for reading and writing files take the form `read*` and `write*`, respectively.

Files can be read using the following subroutines in `Polymatic.pm`:

1. `readLammps`, LAMMPS [data](#) file
2. `readTypes`, *Polymatic* types file (Section [3.2.3](#))
3. `readPolym`, *Polymatic* input script (Section [3.2.4](#))
4. `readPdb`, [PDB](#) file
5. `readGro`, [GRO](#) file
6. `readXyz`, XYZ file
7. `readPsf`, [PSF](#) file, for use in [VMD](#)
8. `readXsd`, XSD file from [Materials Studio](#)

Almost all of these `read*` subroutines take one argument, the name of the file to be read in, and return a hash data structure containing the molecular system (Appendix [A](#)). The only exceptions are `readTypes` and `readPsf`, which takes two arguments, the name of the file to be read in and a molecular system hash already defined. It updates the hash and returns nothing.

Files can be written using the following subroutines in `Polymatic.pm`:

1. `writeLammps`, LAMMPS [data](#) file
2. `writeLmpsTrj`, LAMMPS [dump](#) file
3. `writeTypes`, *Polymatic* types file (Section [3.2.3](#))
4. `writePdb`, [PDB](#) file
5. `writeGro`, [GRO](#) file
6. `writeXyz`, XYZ file
7. `writePsf`, [PSF](#) file, for use in [VMD](#)
8. `writeCar`, CAR file, to open in [Materials Studio](#)
9. `writeMdf`, MDF file, to open in [Materials Studio](#)

All of these `write*` subroutines take two arguments, the name of the file to be written and a hash data structure containing the molecular system (Appendix [A](#)).

B.3 Connectivity Definitions

In the `Polymatic.pm` module, subroutines for defining the connectivity take the form `define*`. They take one argument, a hash data structure containing the molecular system (Appendix [A](#)). Note that bonds must be defined in order to determine other connectivity information.

The connectivity subroutines in `Polymatic.pm` include:

1. `defineAngles`, define all angles in the system
2. `defineDiheds`, define all dihedrals in the system
3. `defineImprops`, define all impropers in the system
4. `defineMols`, define all molecules in the system

Angles and dihedrals are determined as three or four consecutive atoms joined by two or three bonds, respectively. Impropers are determined as a central atom with three other atoms bonded to it. Molecules are defined based on all atoms bonded together.

B.4 Usage

A basic file conversion script, `file_convert.pl`, is provided with the *Polymatic* distribution, which uses command-line flags to call the subroutines from the `Polymatic.pm` module. It is called as:

```
./file_convert.pl <flags>
```

The script is set up to read in a molecular system from a file. These command-line flags begin with ‘i’ and should be followed by the file name of the file to be read in. The possible input flags include:

1. `--ilmp`, read LAMMPS data file (`readLammps`)
2. `--ipdb`, read PDB file (`readPdb`)
3. `--igro`, read GRO file (`readGro`)
4. `--ixyz`, read XYZ file (`readXyz`)
5. `--ixsd`, read XSD file (`readXsd`)
6. `--ipsf`, read PSF file (`readPsf`)
7. `--ityp`, read types file (`readTypes`)

Note that, although multiple files can be read in, only one molecular system can be defined within the script. Any other files read in will overwrite the previously defined molecular system. The exceptions to this are PSF and types files, which modify an already defined system.

After a molecular system is defined, connectivity information can be updated assuming that bonds were included in the system definition. Supported file types in the `Polymatic.pm` module that can include bond definitions are LAMMPS data, PDB, XSD, and PSF files. The possible connectivity flags do not take an option and include:

1. `--mol`, define all molecules (`defineMols`)
2. `--ang`, define all angles (`defineAngles`)
3. `--dih`, define all dihedrals (`defineDiheds`)
4. `--imp`, define all impropers (`defineImprops`)

The script is also set up to write a molecular system to a file. These command-line flags begin with ‘o’ and should be followed by the file name of the file to be written. The possible output flags include:

1. `--olmp`, write LAMMPS data file (`writeLammps`)
2. `--opdb`, write PDB file (`writePdb`)
3. `--ogro`, write GRO file (`writeGro`)
4. `--oxyz`, write XYZ file (`writeXyz`)
5. `--oxsd`, write XSD file (`writeXsd`)
6. `--opsf`, write PSF file (`writePsf`)
7. `--otyp`, write types file (`writeTypes`)
8. `--otrj`, write LAMMPS dump file (`writeLmpsTrj`)
9. `--ocar`, write CAR file (`writeCar`)
10. `--omdf`, write MDF file (`writeMdf`)

Unlike file input, any number of file types can be written. Note that some default values are provided when writing files if necessary data is missing, since different file types do not contain the same information. A warning is printed to standard output if this occurs.

Many of these file types are viewable in VMD, including PDB, GRO, XYZ, and LAMMPS dump files. Any coordinate file can be loaded with a corresponding PSF file containing bonding information. Additionally, a CAR/MDF file pair can be used to load a molecular system in Materials Studio with bonding information.

These command-line flags can be combined as necessary to provide the proper system definitions and file conversions. Below are some examples.

To convert a LAMMPS data file to a PDB file:

```
./file_convert.pl --ilmp in.lmps --opdb out.pdb
```

To convert a PDB file with bonding information to CAR and MDF files:

```
./file_convert.pl --ipdb in.pdb --mol \  
--ocar out.car --omdf out.mdf
```

To convert a Material Studio XSD file to a LAMMPS data file and types file, defining all angles, dihedrals, impropers, and molecules:

```
./file_convert.pl --ixsd in.xsd --mol --ang --dih --imp \  
--olmp out.lmps --otyp types.txt
```

Note: Different file types contain different information about the molecular systems, so translating between file types is not always exact. The subroutines contained in the `Polymatic.pm` module are geared toward use with the *Polymatic* codes, LAMMPS, and VMD. These points should be taken into consideration when using the subroutines in the `Polymatic.pm` module or the included file conversion script, `file_convert.pl`.